Week 4- Sorting Algorithms I

Condensed Study Notes

Generated: 2026-05-25 10:47 UTC

Week 4- Sorting Algorithms I

This week, we begin exploring **Sorting Algorithms**. These are fundamental procedures in computer science designed to arrange a list of items (like numbers or words) into a specific order, such as ascending or descending.

Think of sorting like organizing a deck of cards. You might want to arrange them by suit, then by number. A sorting algorithm is the set of steps you follow to achieve that perfect order.

We'll be looking at how these algorithms work, their efficiency, and their applications.

Overview

Sorting is essentially about rearranging a collection of items, like numbers or words, into a specific, logical order. Think of it like organizing a deck of cards by suit and then by number, or arranging a list of names alphabetically.

This process is called **permuting**, which means changing the order of elements. The collection of items is often stored in something called an **array**, which is like a numbered list or a row of boxes.

Sorting is a very important operation in computer science because many other tasks or programs rely on having data already organized. It's a common first step in solving more complex problems.

Categories of sorting

Sorting algorithms can be broadly categorized into two types based on how much data can be handled by the computer's memory at one time.

- **Internal sorting**: This type of sorting is performed when all the data elements to be sorted can fit into the computer's main memory (RAM) simultaneously. Think of it like organizing a small pile of papers on your desk – you can see and move all of them at once.
- **External sorting**: This is used when the amount of data is too large to fit into memory all at once. The sorting process happens in stages, and the partially sorted pieces are combined later. This is like organizing a massive library's worth of books; you can only work with a section at a time, moving books to different shelves and eventually organizing them all. External sorting relies on secondary storage, like hard drives, to temporarily hold the data and the sorted results.

Some sorting terminologies

As we explore different sorting algorithms, it's helpful to understand some common terms used to describe them.

• **In-place sorting algorithm**: This is a sorting algorithm that sorts data without needing a separate, large area to store the data while it's being sorted. It might use a small, constant amount of extra memory for temporary variables, but it essentially rearranges the data directly within its original location, often by swapping elements.

• Think of it like tidying up a messy desk by moving papers around directly on the desk, rather than moving them all to a separate table to sort them first.

• **Stable sorting algorithm**: A stable sorting algorithm preserves the original order of elements that have the same value. If two identical items are in a certain order in the original list, they will remain in that same order in the sorted list.

• Imagine sorting a list of students by their grades. If two students have the same grade, a stable sort would keep them in the same relative order they appeared in the original class list.

• **Sorting problem**: This refers to the general task of arranging items in a list into a specific order. This order is typically numerical (like 1, 2, 3) or lexicographical (alphabetical, like A, B, C), and can be either non-decreasing (ascending) or non-increasing (descending).

• **Sorting algorithm**: This is the specific set of instructions or steps used to solve a sorting problem, meaning it's the method by which we actually rearrange the elements of a list into the desired order.

Common sorting algorithms

Now that we understand what sorting is and how we can categorize sorting algorithms, we will begin exploring some specific methods. We will be studying six common sorting algorithms in total.

These six algorithms are divided into two groups:

• **Three elementary sorting algorithms**: These are often the first ones taught due to their relative simplicity.

• Selection sort

• Bubble sort

• Insertion sort

Selection sort

Selection sort employs a straightforward, or 'brute-force', method to arrange a list.

Here's how it works:

• In each pass through the list, it finds the smallest item that hasn't been placed in its final sorted position yet.

• Once found, this smallest item is then moved to the very beginning of the unsorted portion of the list.

Think of it like picking out the shortest person in a line of people and asking them to step to the front, then repeating the process with the remaining people to find the next shortest, and so on, until everyone is in order of height.

Selection sort algorithm

Selection sort is a straightforward sorting algorithm that works by systematically finding the smallest element in the unsorted portion of a list and moving it to the beginning of the sorted portion.

Imagine you have a pile of unsorted playing cards, and you want to sort them by number. Selection sort is like picking up the smallest card from the whole pile, placing it aside as your first sorted card, then picking the smallest from the remaining pile for your second sorted card, and so on.

Here's how it operates:

- **Pass 1:** The algorithm scans the entire list to find the smallest element. Once found, it's swapped with the element at the very first position.
- **Pass 2:** It then scans the remaining unsorted part of the list (from the second element onwards) to find the next smallest element. This smallest element is then swapped with the element at the second position.
- **Subsequent Passes:** This process continues. In each pass, the algorithm finds the smallest element in the remaining unsorted sublist and places it at the next available position in the sorted section.

This continues until the entire list is sorted. The list is divided into two parts: a sorted sublist at the beginning and an unsorted sublist at the end. With each pass, one element moves from the unsorted part to the sorted part.

The selection sort is used when

Selection sort is a good choice for sorting in specific situations:

- **When every element needs to be examined:** Selection sort inherently looks at all the remaining unsorted elements in each step to find the smallest one. This means it's not efficient if you only need to check a few items.
- **For small lists:** When you have a small number of items to sort, the overhead of more complex algorithms isn't worth it, and selection sort performs adequately.
- **When swapping is cheap:** The cost of swapping (exchanging the positions of two elements) is not a concern.
- **When writing to memory is expensive:** This is particularly relevant for certain types of memory, like flash memory. Selection sort performs a number of writes (swaps) that is proportional to the square of the number of items (denoted as O(n) in Big O notation). If writing is costly, minimizing writes becomes important, and selection sort does this efficiently compared to some other algorithms where swaps might happen more frequently within the loop.

Bubble sort

Bubble sort is another straightforward method for sorting lists that can be ordered. It works by comparing two elements that are next to each other in the list.

If these adjacent elements are in the wrong order, they are swapped. This process is repeated throughout the list. Imagine you have a list of numbers, and you compare the first two. If the first is larger than the second, you swap them. Then you compare the second and third, and so on. By repeatedly going through the list and swapping out-of-order adjacent items, the largest element in the unsorted part of the list

gradually 'bubbles up' to its correct, final position at the end of the list. This process is repeated until the entire list is sorted.

Bubble sort algorithm

Bubble sort is a simple sorting algorithm that works by repeatedly stepping through the list, comparing each pair of adjacent items, and swapping them if they are in the wrong order. This process is repeated until no swaps are needed, indicating that the list is sorted.

Think of it like gently nudging heavier items towards one end of a line. Each comparison is a nudge, and if an item is out of place (e.g., a heavier person is before a lighter person in a line for a ride), they are swapped. The heaviest unsorted item will gradually 'bubble up' to its correct position at the end of the unsorted portion of the list with each pass.

Key characteristics:

- Compares adjacent elements.
- Swaps elements if they are in the wrong order.
- The largest unsorted element moves to its final position in each pass.

The bubble sort is used when

Bubble sort is a good choice in situations where:

- **Simplicity is key:** You need code that is short and easy to understand.
- **Complexity is not a concern:** The performance overhead of bubble sort isn't a major issue for your specific needs.

Insertion sort

Insertion sort is a sorting algorithm that works by taking an unsorted element in each step and placing it into its correct position within the already sorted part of the list.

It uses a strategy called "decrease and conquer," specifically the "decrease-by-one" technique. This means it assumes that a smaller version of the list has already been sorted. Then, for the next unsorted element, it finds where it belongs among the sorted elements and inserts it there.

Think of it like sorting a hand of playing cards. You pick up one card at a time from a pile and insert it into its correct place in the cards you're already holding, so your hand always remains sorted.

Insertion sort algorithm

Insertion sort builds a sorted list one element at a time. It takes an unsorted element and finds its correct position within the portion of the list that is already sorted, then inserts it there.

Think of it like sorting a hand of playing cards. You pick up one card at a time and insert it into its correct spot in the cards you're already holding, which are kept in sorted order. You compare the new card with the cards in your hand, moving from right to left, until you find the right place for it.

Key characteristics of insertion sort:

- It's an in-place sorting algorithm, meaning it sorts the data within the original array or list without requiring significant extra memory.
- It's a stable sorting algorithm, meaning that elements with equal values maintain their relative order in the sorted output.
- It's efficient for small datasets or nearly sorted datasets.
- Its performance degrades as the dataset size increases.

Insertion Sort is used when

Insertion sort is a practical choice in specific situations:

- **When there are only a few elements left to sort:** If you're in the middle of a larger sorting process and have a small remaining sub-list, insertion sort can efficiently finish the job.
- **When the entire array is small:** For lists that are already small from the start, insertion sort can be a straightforward and effective method.

Summary of complexities of the elementary sorting algorithms

This section summarizes the performance characteristics of the three elementary sorting algorithms we've discussed: Bubble Sort, Selection Sort, and Insertion Sort.

We'll look at their 'time complexity' (how long they take to run), 'swaps or movements' (how much data they move around), 'auxiliary space' (how much extra memory they need), their 'algorithmic paradigm' (the general approach they use), and whether they are 'sorting in place' and 'stable'.

Time Complexity describes how the runtime of an algorithm grows as the number of items (n) increases.

- **Overall:** All three algorithms have an overall time complexity of $O(n^2)$. This means that if you double the number of items, the time taken will roughly quadruple. Think of it like trying to find a specific book in a library by checking every single book on every shelf – as the library grows, the task becomes much, much harder.
- **Best Case:** Bubble Sort and Insertion Sort can perform better in their best-case scenarios, achieving O(n) (linear time), which is much faster. This happens when the list is already sorted or nearly sorted. Selection Sort, however, always takes $O(n^2)$ time, regardless of the initial order.
- **Average and Worst Case:** For all three algorithms, the average and worst-case time complexity is $O(n^2)$.

Swaps or Movements refers to the number of times elements are moved or swapped.

- **Best Case:** All three algorithms perform a minimal number of swaps in their best case (O(1) or constant time).

• **Worst Case:** Bubble Sort and Insertion Sort can perform up to $O(n^2)$ swaps in their worst case. Selection Sort, however, performs at most O(n) swaps, even though it does more comparisons.

Auxiliary Space is the extra memory an algorithm needs beyond the input list itself.

• All three elementary sorting algorithms are very efficient in terms of space, requiring only O(1) auxiliary space. This means they sort 'in-place', using a constant, minimal amount of extra memory, no matter how large the list is.

Algorithmic Paradigm describes the general strategy:

• Bubble Sort and Selection Sort use a **Brute Force** approach, checking many possibilities. Selection Sort also uses **Decrease and Conquer**, as it effectively reduces the unsorted portion of the list with each step.

• Insertion Sort uses **Decrease and Conquer** by building the sorted list one element at a time.

Sorting in Place?

• Yes, all three algorithms sort the list without requiring a significant amount of extra memory.

Stability

• A stable sort means that if two elements have the same value, their original relative order is preserved after sorting. Bubble Sort and Insertion Sort are stable, while Selection Sort is not.